WebLearn Academy

Complete Project Documentation

Interactive Web Learning Platform for Beginners

Section	Description
1. Project Overview	What is WebLearn Academy
2. Technology Stack	Technologies used
3. Project Structure	File organization
4. Backend Architecture	Server, routes, middleware
5. Frontend Architecture	UI, screens, components
6. Database Design	Data storage approach
7. Authentication System	Login, JWT, security
8. Admin Panel	Management features

9. Content System	Topics and lessons
10. How to Run	Setup instructions

1. Project Overview

WebLearn Academy is an interactive web-based learning platform designed to teach web development fundamentals to beginners. The platform provides a structured learning path with 14 topics covering HTML, CSS, JavaScript, HTTP, APIs, Authentication, Databases, and more, culminating in a capstone project.

Key Features:

• Two learning modes: 15-Day Fast Track and 30-Day Full Course

• 5 content sections per topic: Quick Lesson, Deep Dive, Comparison, Interview Q&A;, Exercises

• Progress tracking with streaks and completion statistics

• Admin panel for managing students and viewing analytics

• Modern glassmorphism UI with responsive design

• JWT-based authentication with role-based access control

2. Technology Stack

Layer	Technology	Purpose
Backend	Node.js + Express	Server and API framework
Database	JSON Files	Simple file-based storage
Authentication	JWT (jsonwebtoken)	Secure token-based auth
Password Security	bcrypt	Password hashing
Frontend	HTML5 + CSS3 + JavaScript	Single Page Application
CSS Framework	Bootstrap 5 + Tailwind CSS	Responsive styling
Icons	Font Awesome 6	Icon library
Code Highlighting	Prism.js	Syntax highlighting
Unique IDs	uuid	Generate unique identifiers

3. Project Structure

The project follows a clean separation of concerns with dedicated folders for different purposes:

```
weblearn-academy/
|
|-- server.js              # Main entry point
|-- package.json           # Dependencies
|-- .env                   # Environment variables
|
|-- routes/                # API route handlers
|   |-- auth.js            # Authentication routes
|   |-- topics.js          # Topic content routes
|   |-- progress.js        # Progress tracking routes
|   |-- admin.js           # Admin panel routes
|
|-- middleware/             # Express middleware
|   |-- auth.js            # JWT verification
|   |-- admin.js           # Admin authorization
|   |-- rateLimit.js       # Rate limiting
|
|-- utils/                 # Utility functions
|   |-- db.js              # JSON file operations
|   |-- hash.js            # Password hashing
|
|-- content/               # Learning content
|   |-- index.js           # Topics list
|   |-- modes.js           # Learning modes
|   |-- 01-html/           # HTML topic content
|   |-- 02-css/            # CSS topic content
|   |-- ... (14 topics + capstone)
|
|-- data/                  # User data storage
|   |-- users.json         # User accounts
|   |-- progress.json      # Learning progress
|
|-- public/                # Frontend files
    |-- index.html         # Main HTML file
    |-- styles/
        |-- main.css       # Custom styles
```

4. Backend Architecture

4.1 Server Setup (server.js)

The main server file sets up Express with middleware for JSON parsing, URL encoding, static file serving, and CORS. It mounts four route modules and includes an SPA fallback for client-side routing.

4.2 API Routes

Route	Methods	Description
/api/auth	POST, GET, PUT	Register, login, profile, preferences
/api/topics	GET	List topics, get topic content
/api/progress	GET, POST	Get summary, update progress, log activity
/api/admin	GET, PUT, DELETE	Dashboard, user management, statistics

4.3 Middleware

auth.js - Verifies JWT tokens from the Authorization header. Decodes the token and attaches user info to req.user.

admin.js - Extends auth middleware to check if user has 'admin' role. Returns 403 Forbidden for non-admin users.

rateLimit.js - Prevents abuse by limiting requests per IP. Returns 429 Too Many Requests when limit exceeded.

5. Frontend Architecture

The frontend is a Single Page Application (SPA) built with vanilla JavaScript, Bootstrap 5, and Tailwind CSS. It uses a glassmorphism design theme.

5.1 Screen Structure

Screen	Description	Access
Login Screen	Authentication form with glass card	Public
Customer Dashboard	Stats, progress, topic cards	Customers
Topic View	Content sections with tabs	Customers
Admin Dashboard	Platform statistics	Admins
Admin Students	User management table	Admins
Admin Statistics	Topic completion stats	Admins

5.2 Application State

The App object maintains global state including: JWT token (persisted in localStorage), current user info, topics list, current topic being viewed, active section tab, learning mode, and admin pagination state.

5.3 UI Design Theme

The application uses a glassmorphism design with: frosted glass panels (backdrop-filter: blur), gradient backgrounds (purple to blue), semi-transparent cards, smooth animations, and responsive layout that adapts to mobile devices.

6. Database Design

The application uses a simple JSON file-based database instead of a traditional database system. This approach is suitable for small applications and learning projects.

6.1 Users Collection (users.json)

Each user has: id (UUID), name, email, passwordHash (bcrypt), createdAt, preferences (learningMode, theme), and role (admin/customer).

6.2 Progress Collection (progress.json)

Each user's progress includes: topics (completion status per topic), dailyLog (activity by date), and currentTopicId.

6.3 Content Storage

Learning content is stored in JSON files organized by topic folder. Each topic has 5 content files: quick.json, deep.json, comparison.json, interview.json, and exercises.json.

7. Authentication System

7.1 Registration Flow

1. User submits name, email, and password
2. Server validates input (all fields required, password >= 6 chars)
3. Server checks if email already exists
4. Password is hashed using bcrypt with 10 salt rounds
5. User is saved to users.json with UUID
6. JWT token is created with user id, name, email, role
7. Token is returned to client and stored in localStorage

7.2 Login Flow

1. User submits email and password
2. Server finds user by email
3. bcrypt.compare() verifies password against stored hash
4. If valid, JWT token is created and returned
5. Client stores token and redirects to dashboard

7.3 Protected Routes

Protected routes use the auth middleware which extracts the JWT token from the Authorization header, verifies it, and attaches user info to req.user. Admin routes additionally check for the 'admin' role.

8. Admin Panel

The admin panel provides comprehensive user management and analytics capabilities.

8.1 Admin Dashboard

Shows overview statistics: total students, new registrations this week, completed course count, average progress percentage, and a table of recent students.

8.2 Student Management

• View all students in a paginated table

• Search students by name or email

• Edit student name, email, and role

• Delete students (with confirmation)

• View individual student progress

8.3 Statistics

Shows topic completion counts across all users, helping identify which topics are most challenging or popular.

9. Content System

The learning content is organized into 14 topics plus a capstone project. Each topic has 5 content sections:

Section	Duration	Description
Quick Lesson	5 minutes	Core concepts with key terms
Deep Dive	15 minutes	Detailed explanations and examples
Comparison	10 minutes	JavaScript vs Python side-by-side
Interview Q&A	10 minutes	Common interview questions
Exercises	15 minutes	Coding practice with hints

9.1 Topics List

- 01. HTML - The Structure of the Web
- 02. CSS - Styling Your Pages
- 03. JavaScript - Making Pages Interactive
- 04. HTTP/HTTPS - How the Web Talks
- 05. API Gateway & Rate Limiting
- 06. Routing, REST APIs & GraphQL
- 07. Authentication - JWT, Sessions & OAuth
- 08. Cookies vs LocalStorage
- 09. Databases & CRUD Operations
- 10. Error Handling
- 11. Loading States & Caching
- 12. Deployment - Putting Your App Online
- 13. Modern HTML & CSS - Flexbox, Grid & Variables
- 14. Modern JavaScript - ES6+, Async/Await & Fetch
- Capstone - DevBlog Platform

10. How to Run

10.1 Prerequisites

• Node.js (v14 or higher)

• npm (comes with Node.js)

• A modern web browser

10.2 Installation

Step 1: Navigate to project directory

```
cd weblearn-academy
```

Step 2: Install dependencies

```
npm install
```

10.3 Running the Server

Start the server:

```
npm start
```

Server will start at http://localhost:2007

10.4 Test Accounts

Role	Email	Password
Admin	admin@weblearn.com	admin123
Customer	test@test.com	customer123

Appendix: API Reference

Authentication Endpoints

Method	Endpoint	Body	Description
POST	/api/auth/register	name, email, password	Create account
POST	/api/auth/login	email, password	Login
GET	/api/auth/profile	-	Get profile
PUT	/api/auth/preferences	learningMode, theme	Update prefs

Topics Endpoints

Method	Endpoint	Description
GET	/api/topics	List all topics
GET	/api/topics/:id	Get topic content

Progress Endpoints

Method	Endpoint	Description
GET	/api/progress/summary	Get progress summary
POST	/api/progress/topic/:id	Update topic progress
POST	/api/progress/topic/:id/interview	Record interview attempt
POST	/api/progress/topic/:id/exercise	Record exercise attempt
POST	/api/progress/log	Log daily activity

Admin Endpoints

Method	Endpoint	Description
GET	/api/admin/dashboard	Get dashboard stats

GET	/api/admin/users	List users (paginated)
GET	/api/admin/users/:id	Get user details
PUT	/api/admin/users/:id	Update user
DELETE	/api/admin/users/:id	Delete user
GET	/api/admin/progress	Get all progress
GET	/api/admin/stats	Get platform statistics